

POST MORTEM

Aplicação da Classe Pilha

Problema II - Torre de Hanói

MAE014 - Tópico em Ciência de Dados B: Estrutura de Dados

Integrantes	DRE
Alanna Figueiredo Simões	120053919
Artur de Melo Brito	117224234
Emanuel Cardoso	122033151
Felipe Rodrigues Schoeffer	123311188

Code Review

Falhas de Eficiência

A partir do código bruto foram identificados alguns pontos a serem alterados:

1. A apresentação dos pinos não era fixa.

Durante a recursividade, os papéis de pino inicial, intermediário e destino são trocados entre as chamadas das funções. Caso a impressão utilize diretamente essas variáveis, a posição visual dos pinos poderia mudar durante a execução.

```
def mostra_pinos(pino_inicial, pino_intermediario, pino_destino,
movimentos):
    print()
    print(f"Posição: {movimentos} movimentos")
    print()

    pinos = [
        pino_inicial,
        pino_intermediario,
        pino_destino
    ]

    nomes = [
        "Pino inicial",
        "Pino intermediário",
        "Pino destino"
    ]

    for nome, pino in zip(nomes, pinos):
        elementos = []

        for i in range(pino.tamanho()):
            elementos.append(str(pino.dados[i]))

        print(f"{nome}: [ {' '.join(elementos)} ]")

    print()
```

2. Não houve tratamento de erro para entradas não inteiras nas variáveis N e M.

Entradas de outros tipos de dados, como string e ponto flutuante, provocavam um `ValueError` diretamente no `int(input())`, encerrando o programa.

```
N = int(input("Digite a quantidade de discos N: "))

while N <= 0:
    print("N deve ser maior que zero.")
    N = int(input("Digite a quantidade de discos N: "))
```

```
M = int(input("Digite a quantidade de movimentos M entre visualizações: "))

while M <= 0:
    print("M deve ser maior que zero.")
    M = int(input("Digite a quantidade de movimentos M entre visualizações: "))
```

3. Foi necessário verificar a utilização da classe Pilha.

A solução da Torre de Hanói depende diretamente de `empilha()` e `desempilha()`. Na implementação fornecida, o método `desempilha()` não reduzia a quantidade de elementos, apenas retornava o elemento que estava no topo, sem o excluir apenas direcionando a referência de fim da fila para o penúltimo elemento. Isso gerava uma apresentação dos pinos com elementos que não faziam parte do pino.

```
def desempilha(self):
    if self.pilha_esta_vazia():
        raise PilhaVaziaErro("A pilha está vazia.")

    # Decrementa o ponteiro do topo e retorna o elemento
    self.quantidade -= 1
    return self.dados[self.quantidade]
```

Complexidade do algoritmo

O algoritmo bruto apresenta uma complexidade de tempo exponencial, consequência do número de movimentos necessários a serem feitos na recursividade.

$$T(n) = 2^n - 1 \rightarrow O(2^n)$$

Em relação ao espaço ocupado, a maior profundidade da pilha é n , mesmo no espaço auxiliar devido à recursão, quanto ao armazenamento. Assim temos:

$$O(n)$$